Lecture 14. Model Building

Diamonds and daily flights: building models step by step

Contents

14.1 Building models step by step **2**

14.2 Why are low-quality diamonds more expensive? **2**

14.2.1 Price and carat 2

14.2.2 A more complicated model 3

14.3 What affects the number of daily flights? **4**

14.3.1 Day of week 5

14.3.2 Seasonal Saturday effect 5

14.3.3 Time of year: an alternative approach 7

14.4 Selected exercises **7**

14.1 Building models step by step

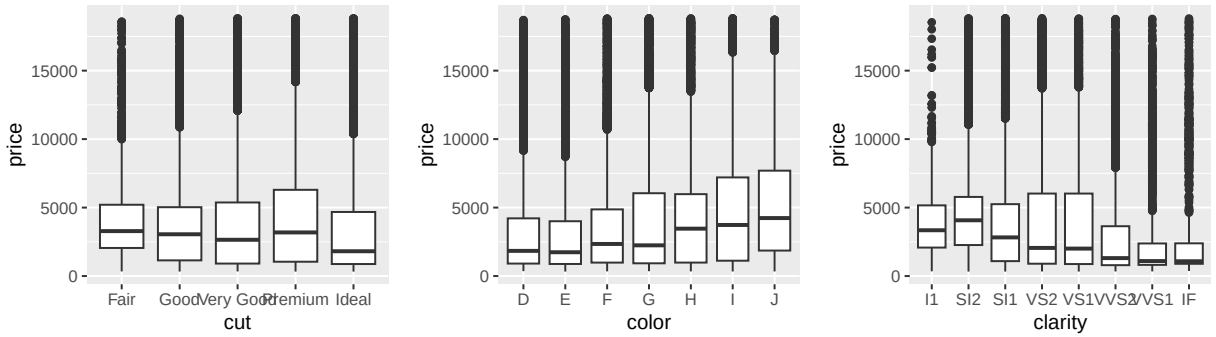
Motivation. Real data overlay patterns; peel them off one at a time through the residuals.

Plot a pattern, model it, study the residuals for the next; implicit knowledge ("prices depend on size") becomes an explicit model. Packages: `modelr`, `nycflights13`, `lubridate`.

14.2 Why are low-quality diamonds more expensive?

Poor cut, colour (J worst) and clarity (I1) have higher median prices (Lecture 4):

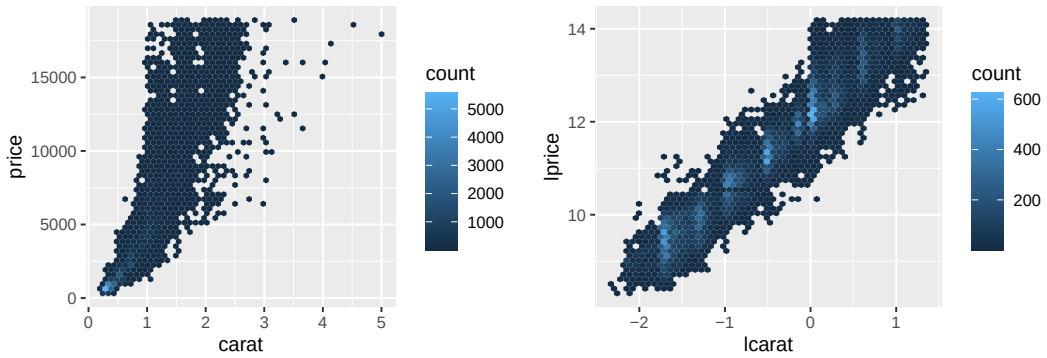
```
ggplot(diamonds, aes(cut, price)) + geom_boxplot()
ggplot(diamonds, aes(color, price)) + geom_boxplot()
ggplot(diamonds, aes(clarity, price)) + geom_boxplot()
```



14.2.1 Price and carat

Confounding: `carat` drives price, and low-quality diamonds are larger. Remove carat first: keep ≤ 2.5 carats (99.7%) and log both, which linearises:

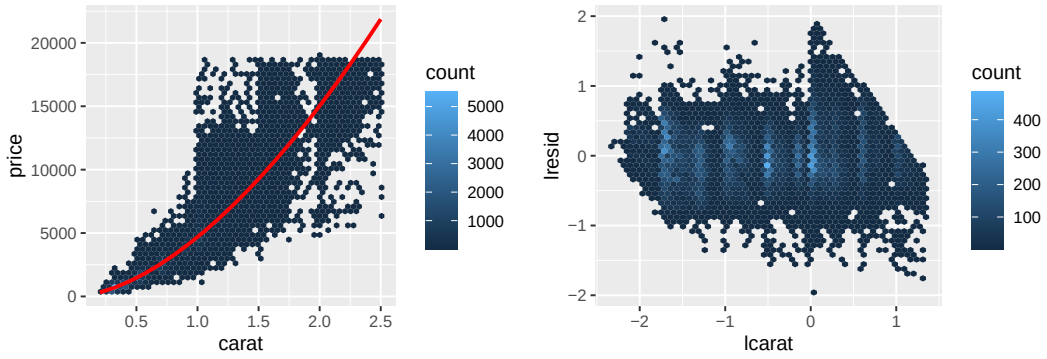
```
diamonds2 <- diamonds %>%
  filter(carat <= 2.5) %>%
  mutate(lprice = log2(price), lcarat = log2(carat))
ggplot(diamonds, aes(carat, price)) + geom_hex(bins = 50)
ggplot(diamonds2, aes(lcarat, lprice)) + geom_hex(bins = 50)
```



Fit on the log scale, back-transform:

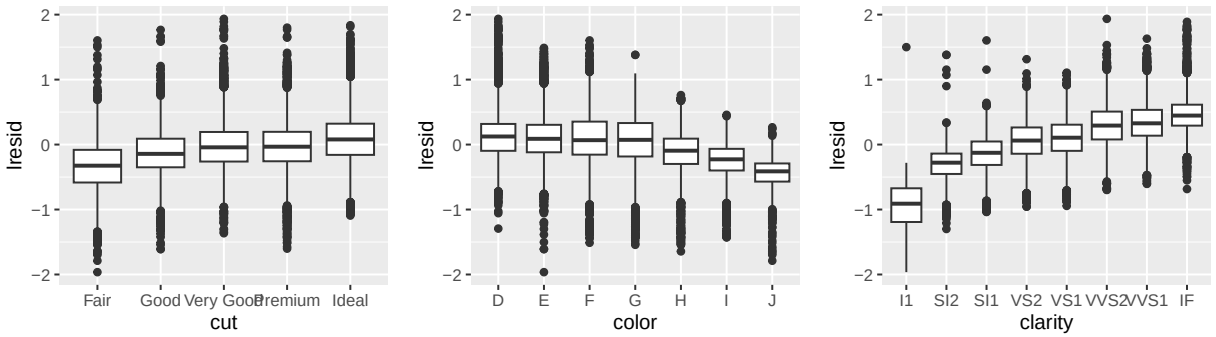
```
mod_diamond <- lm(lprice ~ lcarat, data = diamonds2)
grid <- diamonds2 %>%
  data_grid(carat = seq_range(carat, 20)) %>%
```

```
  mutate(lcarat = log2(carat)) %>%
  add_predictions(mod_diamond, "lprice") %>%
  mutate(price = 2 ^ lprice)
ggplot(diamonds2, aes(carat, price)) +
  geom_hex(bins = 50) +
  geom_line(data = grid, colour = "red", linewidth = 1)
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond, "lresid")
ggplot(diamonds2, aes(lcarat, lresid)) + geom_hex(bins = 50)
```



Large diamonds fall below the curve (no price exceeds \$19 000). Residuals no longer depend on carat and show the expected quality order:

```
ggplot(diamonds2, aes(cut, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(color, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(clarity, lresid)) + geom_boxplot()
```



On the $\log_2$ scale, -1 is half the carat-based price, $+1$ twice; relative price rises with quality.

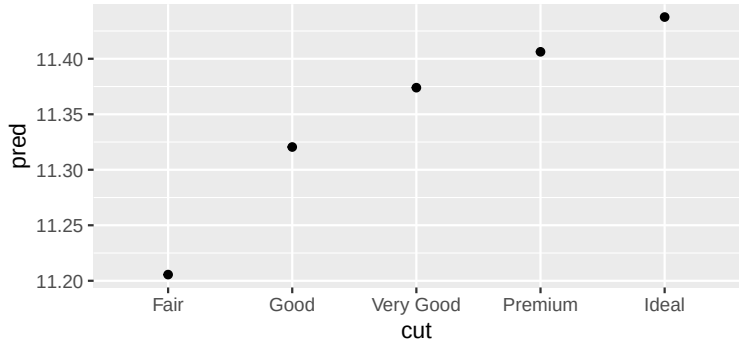
14.2.2 A more complicated model

Add colour, cut and clarity:

```
mod_diamond2 <- lm(lprice ~ lcarat + color + cut + clarity, data = diamonds2)
```

One effect at a time: `data_grid(cut, .model = mod_diamond2)` sets the others to typical values (median or modal level):

```
grid <- diamonds2 %>%
  data_grid(cut, .model = mod_diamond2) %>%
  add_predictions(mod_diamond2)
ggplot(grid, aes(cut, pred)) + geom_point()
```



Some residuals exceed $|r| > 1$ (a factor of two): data errors or bargains?

```
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond2, "lresid2")
diamonds2 %>%
  filter(abs(lresid2) > 1) %>%
  add_predictions(mod_diamond2) %>%
  mutate(pred = round(2 ^ pred)) %>%
  select(price, pred, carat:table, x:z) %>%
  arrange(price)
```

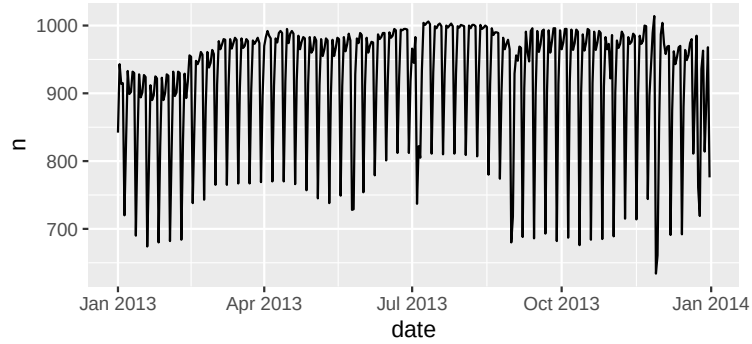
```
## # A tibble: 16 x 11
##   price  pred carat cut     color clarity depth table     x     y     z
##   <int> <dbl> <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  1013   264  0.25 Fair    F     SI2      54.4    64  4.3   4.23  2.32
## 2  1186   284  0.25 Premium G     SI2      59      60  5.33  5.28  3.12
## 3  1186   284  0.25 Premium G     SI2      58.8    60  5.33  5.28  3.12
## 4  1262  2644  1.03 Fair    E     I1       78.2    54  5.72  5.59  4.42
## 5  1415   639  0.35 Fair    G     VS2      65.9    54  5.57  5.53  3.66
## # i 11 more rows
```

14.3 What affects the number of daily flights?

Motivation. Weekly cycles, seasons and holidays, each seen after removing the last.

Flights per day (365 rows):

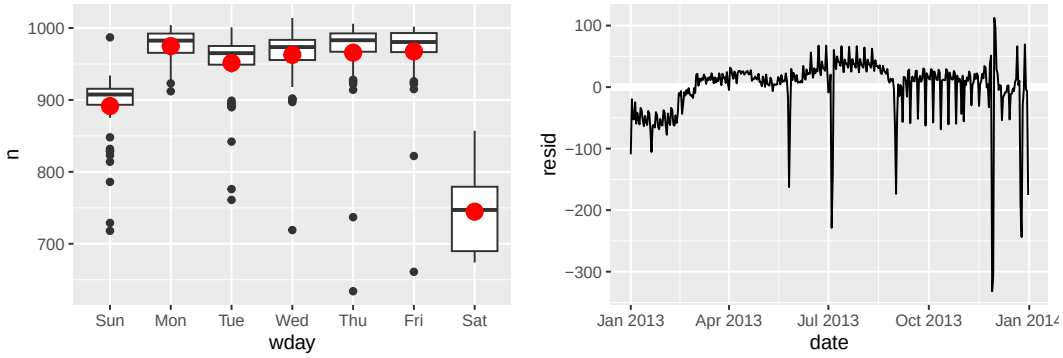
```
daily <- flights %>%
  mutate(date = make_date(year, month, day)) %>%
  group_by(date) %>%
  summarise(n = n())
ggplot(daily, aes(date, n)) + geom_line()
```



14.3.1 Day of week

Business travel: fewer flights at weekends, especially Saturdays. `n ~ wday` predicts weekday means:

```
daily <- daily %>% mutate(wday = wday(date, label = TRUE))
mod <- lm(n ~ wday, data = daily)
grid <- daily %>% data_grid(wday) %>% add_predictions(mod, "n")
ggplot(daily, aes(wday, n)) +
  geom_boxplot() +
  geom_point(data = grid, colour = "red", size = 4)
daily <- daily %>% add_residuals(mod)
daily %>%
  ggplot(aes(date, resid)) +
  geom_ref_line(h = 0) +
  geom_line()
```



Residuals show what it misses:

- from June a regular pattern: Saturdays (coloured by weekday) are badly fit, more in summer, fewer in autumn;
- `filter(resid < -100)`: US holidays (New Year, 4 July, Thanksgiving, Christmas);
- a smooth trend, fewer in January and December, more in summer (`geom_smooth(span = 0.20)`).

```
daily %>% filter(resid < -100)
```

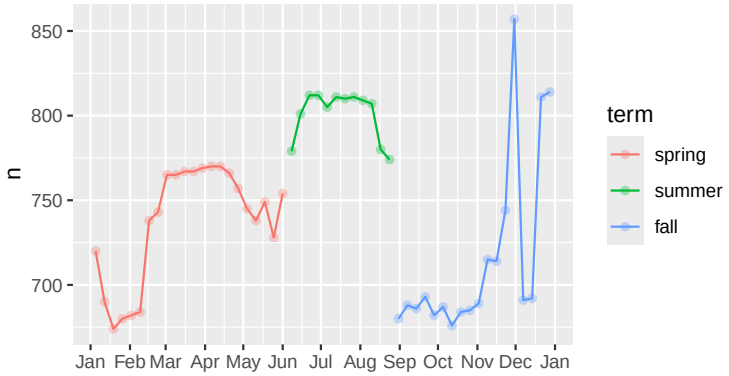
```
## # A tibble: 11 x 4
##   date           n wday  resid
##   <date>     <int> <ord> <dbl>
## 1 2013-01-01   842 Tue   -109.
## 2 2013-01-20   786 Sun   -105.
## 3 2013-05-26   729 Sun   -162.
## 4 2013-07-04   737 Thu   -229.
## 5 2013-07-05   822 Fri   -145.
## # i 6 more rows
```

14.3.2 Seasonal Saturday effect

Saturdays alone suggest school terms (summer-holiday travel); a `term` with breaks read off the plot:

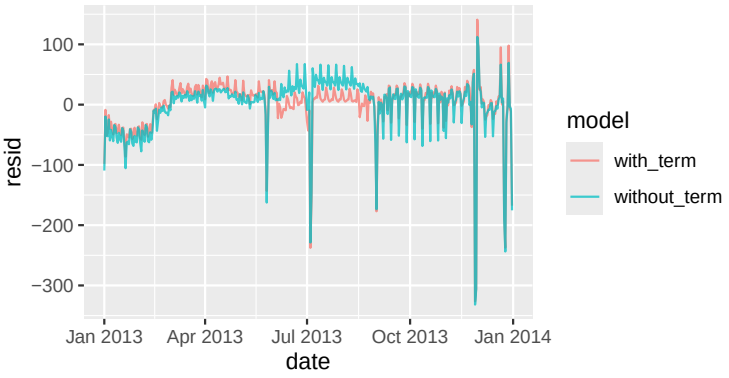
```
term <- function(date) {
  cut(date,
      breaks = ymd(20130101, 20130605, 20130825, 20140101),
      labels = c("spring", "summer", "fall"))
```

```
}
daily <- daily %>% mutate(term = term(date))
daily %>%
  filter(wday == "Sat") %>%
  ggplot(aes(date, n, colour = term)) +
  geom_point(alpha = 1/3) +
  geom_line() +
  scale_x_date(NULL, date_breaks = "1 month", date_labels = "%b")
```



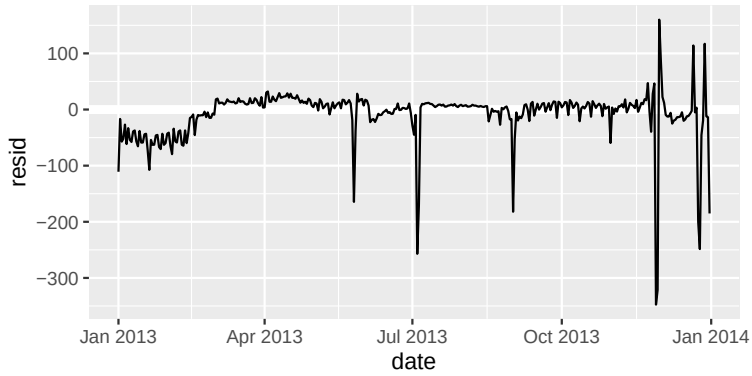
(Labels depend on the locale: a Korean system, as on the slides, returns Korean abbreviations for the filter.) `n ~ wday * term` improves the fit:

```
mod1 <- lm(n ~ wday, data = daily)
mod2 <- lm(n ~ wday * term, data = daily)
daily %>%
  gather_residuals(without_term = mod1, with_term = mod2) %>%
  ggplot(aes(date, resid, colour = model)) +
  geom_line(alpha = 0.75)
```



Cell *means* are pulled by holidays; robust `MASS::rlm()` downweights them, fits the regular pattern better and leaves holidays as large residuals:

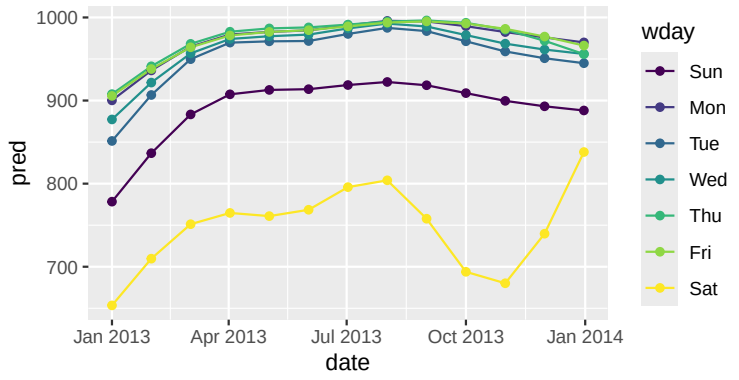
```
mod3 <- MASS::rlm(n ~ wday * term, data = daily)
daily %>%
  add_residuals(mod3, "resid") %>%
  ggplot(aes(date, resid)) +
  geom_hline(yintercept = 0, linewidth = 2, colour = "white") +
  geom_line()
```



14.3.3 Time of year: an alternative approach

A flexible alternative: a natural spline in date, interacting with weekday, a smooth curve per weekday:

```
library(splines)
mod <- MASS::rlm(n ~ wday * ns(date, 5), data = daily)
daily %>%
  data_grid(wday, date = seq_range(date, n = 13)) %>%
  add_predictions(mod) %>%
  ggplot(aes(date, pred, colour = wday)) +
  geom_line() + geom_point()
```



The Saturday pattern reappears unprompted: two approaches, one signal.

Explicit versus flexible models [Supplement]

`term`: a hypothesis in three parameters per weekday, interpretable, but its breaks came from the plot (do not test on the same data). Spline: only smoothness, more parameters, less guesswork. Their agreement suggests the Saturday pattern is real.

14.4 Selected exercises

Motivation. Log–log lines, back-transformed predictions, and too many parameters.

Exercise 14.1 (R4DS Exercise 24.2.3.2)

If $\log_2(\text{price}) = a_0 + a_1 \log_2(\text{carat})$, what does that say about the relationship between price and carat?

Solution.

Step 1. price $= 2^{a_0}\,\text{carat}^{a_1}$, a power law; the base does not affect a_1 ($\log_2 u = \log u / \log 2$ scales both sides).

Step 2. Carat $\times c$ gives price $\times c^{a_1}$ (doubling: 2^{a_1}). $d\log(\text{price})/d\log(\text{carat}) = a_1$: 1% heavier costs about a_1% more (elasticity).

Step 3. If $a_1 > 1$, price per carat $2^{a_0}\text{carat}^{a_1-1}$ rises with size. A $\log_2$ residual r means 2^r times the prediction. ■

Exercise 14.2 (back-transforming predictions)

Suppose $\log_2(\text{price}) = \mu(x) + \epsilon$ with $\epsilon \sim N(0, \sigma^2)$ independent of x. Show that $2^{\mu(x)}$, the back-transformed prediction used in the lecture, is the conditional *median* of price, and that the conditional mean is $2^{\mu(x)} \exp\{(\log 2)^2\sigma^2/2\}$.

Solution [Supplement].

Step 1. $u \mapsto 2^u$ is strictly increasing: $P(\text{price} \le 2^{\mu(x)} \mid x) = P(\epsilon \le 0) = \frac{1}{2}$, the median.

Step 2. Mean: $\mathbb{E}[2^\epsilon] = \mathbb{E}[e^{c\epsilon}]$, $c = \log 2$; completing the square,

$$\begin{aligned}\mathbb{E}[e^{c\epsilon}] &= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(cu - \frac{u^2}{2\sigma^2}\Big) du \\ &= e^{c^2\sigma^2/2} \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(-\frac{(u - c\sigma^2)^2}{2\sigma^2}\Big) du,\end{aligned}$$

the last integral being 1 (the $N(c\sigma^2, \sigma^2)$ density).

Step 3. So $\mathbb{E}[\text{price} \mid x] = 2^{\mu(x)} e^{(\log 2)^2\sigma^2/2} >$ median: back-transforms underestimate means by this factor but match typical (median) prices. ■

Exercise 14.3 (R4DS Exercises 24.3.5.5 and 24.3.5.6)

(a) What happens if you fit a day-of-week effect that varies by month, `n ~ wday * month`, with `month` a factor? Why is this not very helpful?
(b) What would you expect the model `n ~ wday + ns(date, 5)` to look like?

Solution.

(a)

Step 1. The column space is spanned by the $7 \times 12 = 84$ cell indicators: 84 coefficients for 365 days. As in Exercise 13.2, the RSS separates over cells, so each is fitted by its mean.

Step 2. Cells hold four or five days, so one holiday moves a mean a lot: low bias, high variance, poor generalisation. One parameter per cell also misses the real structure (a Saturday effect changing with term), which `wday * term` expresses in a handful.

(b) Additive: one smooth 5-df seasonal curve shifted by a constant weekday effect, seven parallel curves. The seasonal trend is captured, not the summer Saturday rise, which needs `wday * ns(date, 5)`; Saturday residuals keep the term pattern. ■